

TD Système d'Exploitation II – Fonction fork()

Institut Supérieur d'Informatique et de Multimédia de Gabès

Niveau : CPI2

Année Universitaire : 2025/2026

Rappel sur la fonction fork()

La fonction `fork()` permet à un processus (père) de créer un nouveau processus (fils) qui est une copie conforme du processus courant.

— **Valeurs de retour :**

- -1 en cas d'échec
- 0 dans le processus fils
- Le PID du fils dans le processus père
- Les deux processus s'exécutent concurrentiellement
- Espaces mémoire complètement séparés

Exercice 1 : Analyse de programmes avec fork()

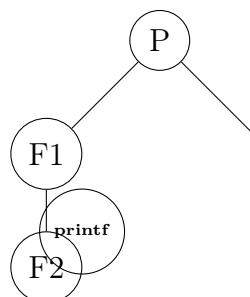
Combien de lignes "hello !" imprime chacun des programmes suivants ?

Programme 1

```
1 if (fork() == 0)
2     if (fork() != 0)
3         printf("hello!\n");
```

Réponse : 1 ligne

Arbre généalogique :

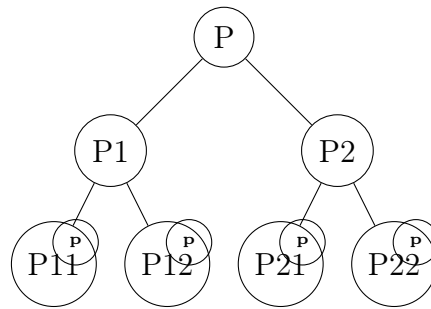


Programme 2

```
1 fork();
2 fork();
3 printf("hello!\n");
```

Réponse : 4 lignes

Arbre généalogique :



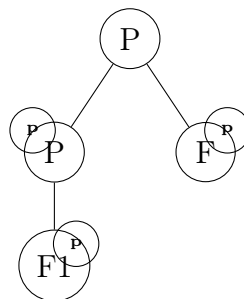
(p = printf)

Programme 3

```
1 if (fork() != 0)
2     fork();
3 printf("hello!\n");
```

Réponse : 3 lignes

Arbre généalogique :



Exercice 2 : Expression complexe

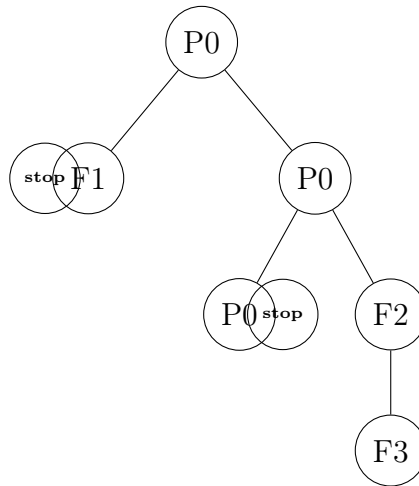
1. Nombre de processus

Combien de processus engendre l'évaluation de la commande :

```
1 fork() && (fork() || fork());
```

Réponse : 5 processus (en comptant le processus initial)

2. Arbre généalogique



Déroulement détaillé :

1. Premier `fork()` crée P0 et F1
2. F1 reçoit 0 → expression `0 && ...` = faux → fin
3. P0 reçoit `PID>0` → évalue `(fork() || fork())`
4. Deuxième `fork()` crée P0 et F2
 - P0 reçoit `PID>0` → `(vrai || ...)` = vrai → court-circuit → fin
 - F2 reçoit 0 → évalue troisième `fork()`
5. Troisième `fork()` crée F2 et F3

Exercice 3 : Création d'arbres généalogiques

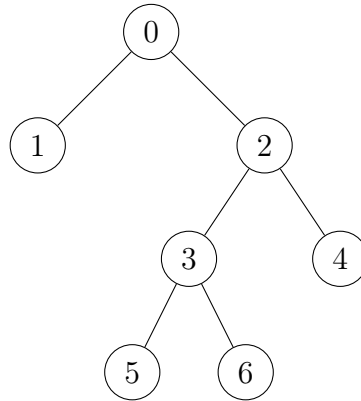
Arbre 1 : Chaîne linéaire



```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int main() {
6     pid_t pid;
7     for (int i = 0; i < 5; i++) {
8         pid = fork();
9         if (pid > 0) { // Pere
10             wait(NULL); // Attend son fils
11             break;      // Sort de la boucle
12         }
13         // Fils continue a creer le suivant
14     }
15     printf("Processus (PID=%d, PPID=%d)\n", getpid(), getppid());
16     return 0;
17 }
```

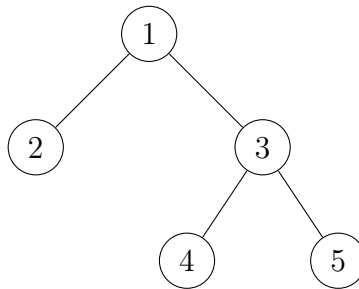
Listing 1 – Programme pour l'arbre linéaire

Arbre 2 : Arbre binaire partiel



```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int main() {
6     printf("Processus 0 (PID=%d)\n", getpid());
7
8     // Creation processus 1
9     if (fork() == 0) {
10         printf("Processus 1\n");
11         return 0;
12     }
13
14     // Creation processus 2
15     if (fork() == 0) {
16         printf("Processus 2\n");
17
18         // Creation processus 3
19         if (fork() == 0) {
20             printf("Processus 3\n");
21
22             // Creation processus 5 et 6
23             fork(); // 5
24             if (fork() == 0) { // 6
25                 printf("Processus 6\n");
26                 return 0;
27             }
28             printf("Processus 5\n");
29             wait(NULL); wait(NULL);
30             return 0;
31         }
32
33         // Creation processus 4
34         if (fork() == 0) {
35             printf("Processus 4\n");
36             return 0;
37         }
38
39         wait(NULL); wait(NULL);
40         return 0;
41     }
42
43     wait(NULL); wait(NULL);
44     return 0;
45 }
```

Arbre 3 : Arbre symétrique



```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int main() {
6     printf("Processus 1\n");
7
8     // Branche gauche
9     if (fork() == 0) {
10         printf("Processus 2\n");
11         return 0;
12     }
13
14     // Branche droite
15     if (fork() == 0) {
16         printf("Processus 3\n");
17
18         fork(); // Processus 4
19         if (fork() == 0) { // Processus 5
20             printf("Processus 5\n");
21             return 0;
22         }
23         printf("Processus 4\n");
24         wait(NULL);
25         return 0;
26     }
27
28     wait(NULL); wait(NULL);
29     return 0;
30 }
```

Listing 3 – Programme pour l'arbre symétrique

Exercice 4 : 10 processus fils

Écrire un programme qui crée 10 processus fils. Chacun affiche dix fois son numéro d'ordre.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
```

```

6 int main() {
7     int i, j;
8     pid_t pid;
9
10    printf("Pere (PID=%d) va creer 10 fils...\n", getpid());
11
12    for (i = 0; i < 10; i++) {
13        pid = fork();
14
15        if (pid < 0) {
16            perror("fork");
17            exit(1);
18        }
19
20        if (pid == 0) {
21            // Processus fils i
22            for (j = 0; j < 10; j++) {
23                printf("Fils %d - Affichage %d/10\n", i, j+1);
24            }
25            exit(0); // Fin du fils
26        }
27    }
28
29    // Pere attend tous les fils
30    for (i = 0; i < 10; i++) {
31        wait(NULL);
32    }
33
34    printf("Tous les fils ont termine.\n");
35    return 0;
36 }

```

Listing 4 – Programme avec 10 processus fils

Conseils pratiques

- **Toujours vérifier** la valeur de retour de `fork()`
- **Utiliser** `getpid()` et `getppid()` pour identifier les processus
- **Gérer les erreurs** avec `perror()` et `exit()`
- **Synchroniser** avec `wait()` ou `waitpid()`
- **Dessiner l'arbre** avant d'écrire le code
- **Tester** avec un petit nombre de processus d'abord

Points clés à retenir

1. `fork()` duplique le processus appelant
2. Le père et le fils ont des espaces mémoire indépendants
3. La différence père/fils se fait sur la valeur de retour
4. L'ordre d'exécution n'est pas garanti (dépend du scheduler)
5. Chaque `fork()` double le nombre de processus (sans condition)

**Fin du TD – Bonne continuation dans l'apprentissage des systèmes
d'exploitation !**